This keyword

In JavaScript, this depends on **who is calling the function**, not where the function is written.

That is the core rule.

this is not about where the function is created.
this is about **how the function is called**.

In JavaScript, this depends on execution context.

1️⃣ Global Context

```
console.log(this);
```

In browser → window
In Node → global (or {} in modules);

2️⃣ Inside a Regular Function

```
function test() {
  console.log(this);
}

test();
```

In non-strict mode → global object
In strict mode → undefined

3️⃣ Inside an Object Method

```
const user = {
  name: "Prajwal",
  greet() {
```

```
    console.log(this.name);
  }
};

user.greet();
```

Here:

this refers to user

Because user called the function

⚡ Inside a Class

```
class Person {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(this.name);
  }
}

const p = new Person("Rahul");
p.speak();
```

Here:

- this refers to the instance

5️⃣ Arrow Function → No Own this

```
const user = {
  name: "Prajwal",
  greet: () => {
    console.log(this.name);
  }
};
```

👉 this is NOT user
👉 It takes this from outer scope

Rule:
Arrow function → inherits this

Hence o/p undefiend

6. 5️⃣ Inside Event Listener

```
button.addEventListener("click", function () {
  console.log(this);
});
```

👉 this = button element

Rule:
Event handler → this = element that triggered it

🎯 How To Find What this Refers To

When confused, ask:

1. Is this inside an object method?
2. Is it called with new?
3. Is it an arrow function?
4. How is this function being called?

⚡ Quick Memory Trick

object.method() → this = object
new Something() → this = new object
arrow function → no own this
normal function → global/undefined

```
// problem 1
const obj={
   name:"Prajwal",
   greet: function(){
      function inner(){
         console.log(this.name);
      }

      inner();
   }
}

obj.greet();

// inner is called as normal function hence this is global or undefined

const obj2={
   name:"Prajwal",
   greet:function(){
      const inner=()=>{
         console.log(this.name);
```

```
        }

        inner();
    }
}

obj2.greet();
```

// arrow function does not have their own this that why it looks for parent this borrow which is greet function.